



INSTITUTO FEDERAL
São Paulo
Campus São Carlos

Introdução a Listas

APR1 – Algoritmos e Programação

Profa. Dra. Eloize Seno



INSTITUTO FEDERAL
São Paulo
Campus São Carlos

Introdução

- As variáveis de um programa podem ser **simples** ou **compostas**.
- Uma **variável simples** é capaz de armazenar **apenas um valor** em um dado momento.
- Exemplos de variáveis simples:
 - idade, salario, data_nascimento, nome_cliente, endereço, etc.
- Uma **variável composta** pode armazenar um **conjunto de valores** ao mesmo tempo.
- Por exemplo, uma **variável composta** é capaz de armazenar o **nome**, **endereço**, **salario** e o **total_horas_trabalhadas** de um funcionário de uma empresa.
 - Tudo em uma única variável!

O que é uma lista?

- Variável composta heterogênea que armazena um conjunto de valores (variáveis).
- Os valores que compõem uma lista são chamados de **elementos**.
- Por ser heterogênea, os elementos de uma lista podem ser de **tipos diferentes** (inclusive podem ser outras listas).
- Cada elemento em uma Lista é identificado por meio de um **índice**.

Definição de listas em Python

- Uma lista em Python é declarada da seguinte forma:
 - `Nome_lista = [valor1, valor2, ..., valorN]`
- Exemplo de lista:
 - `L = [145, 'Maria da Silva', 'Rua das Gardêneas', 38, 18562.60]`
- Uma lista vazia é representada apenas por []. Exemplo:
 - `L = [] # lista vazia`
- Para acessar um elemento qualquer da lista, usamos o **índice** que indica a posição do elemento na lista.

Acessando um elemento de uma lista

- Para acessar um elemento qualquer de uma lista, usamos o nome da lista e o valor do índice do elemento entre colchetes (**operador de indexação**).
 - Em Python, o primeiro elemento de uma lista está na posição **zero**.
- Por exemplo, na lista L a seguir o elemento 145 encontra-se na posição de índice **zero**, enquanto que o elemento 38 está na posição de índice **3**.

L = [145, 'Maria da Silva', 'Rua das Gardêneas', 38, 18562.60]

0

1

2

3

4

- Exemplos:

```
>>>L = [145, 'Maria da Silva', 'Rua das Gardêneas', 38, 18562.60]
```

```
>>> print(L[1])
```

Maria da Silva

```
>>> print(L[3])
```

38

Acessando um elemento de uma lista (cont.)

- Índices negativos podem ser usados para acessar elementos da direita para a esquerda de uma lista
- O índice -1 acessa o **último** elemento de uma lista

- Exemplos:

```
>>> print(L[-1])
```

```
18562.6
```

```
>>> print(L[-3])
```

```
Rua das Gardêneas
```

Modificando os elementos de uma lista

- As **listas** no Python **são mutáveis**, podendo ser alteradas a qualquer momento.
- Por exemplo, se quisermos alterar o elemento da posição de índice 3 da lista L (slide anterior), podemos fazer a seguinte atribuição:
 - `L[3] = 'São Carlos'`

- Ao imprimir a nova lista, o resultado será:

```
>>>L[3]= 'São Carlos'
```

```
>>> print(L)
```

```
[145, 'Maria da Silva', 'Rua das Gardênias', 'São  
Carlos', 18562.6]
```

Percorrendo uma lista por índice

- Também podemos percorrer cada elemento de uma lista usando o operador de indexação (com **for** ou **while**).
- Percorrendo uma lista com **for** por índice:

```
cavaleiros = ['guerra', 'fome', 'peste', 'morte']  
for i in range(len(cavaleiros)):  
    print(cavaleiros[i])
```

- Percorrendo uma lista com **while** (somente por índice):

```
cavaleiros = ['guerra', 'fome', 'peste', 'morte']  
i = 0  
while i < len(cavaleiros):  
    print(cavaleiros[i])  
    i = i + 1
```

Principais operações com listas

- Verificar se um elemento pertence ou não a uma lista
- Verificar o comprimento de uma lista
- Inserir um novo elemento em uma lista
- Remover um elemento de uma lista
- Concatenar listas

Pertinência em uma lista

- **in** e **not in** são operadores booleanos ou lógicos que testam a pertinência de um elemento em uma lista
 - O resultado é sempre True ou False

- Exemplo:

```
frutas = ["maça", "laranja", "banana", "cereja"]
```

```
print("maça" in frutas) # imprime TRUE
```

```
print("pera" in frutas) # imprime FALSE
```

```
print("morango" not in frutas) # imprime TRUE
```

Comprimento de Listas

- O comprimento de uma lista, ou o número de itens que a compõem, pode ser obtido a partir da função **len()**
- Exemplo: o código abaixo indica que a variável lista contém 7 elementos:

```
lista=[3,67,"gato", 56, 808, 3.14, False]  
print(len(lista))
```

Saída: 7

Concatenação

- O operador **+** concatena listas

- Exemplos:

```
>>>print([1, 2] + [3, 4])  
[1, 2, 3, 4]
```

```
>>>frutas = ["maca", "laranja", "banana", "cereja"]  
>>>print(frutas + [6, 7, 8, 9])  
['maca', 'laranja', 'banana', 'cereja', 6, 7, 8, 9]
```

```
lista_orig = [45,32,88]  
lista_orig = lista_orig + ["cachorro"]  
print(lista_orig)
```

```
[45, 32, 88, 'cachorro']
```

Inserir um elemento no final de uma lista

- O método `append` adiciona um novo elemento no final de uma lista

- Exemplo:

```
>>>frutas = ["maca", "laranja", "banana", "cereja"]
```

```
>>>frutas.append("acerola")
```

```
>>>print(frutas)
```

```
['maca', 'laranja', 'banana', 'cereja', 'acerola']
```

- É importante observar que com `append` a lista original é modificada, porém o seu id não se altera
 - É possível ver isto usando a função `id()`

A função id()

- Python possui uma função nativa (*build-in*) que recebe um objeto como argumento e retorna o seu id
- O id é usualmente um número inteiro muito grande, pois corresponde ao endereço de memória da variável
- Exemplo:

```
>>> lista = [4, 5, 6]
>>> print(id(lista))
47295424
```

Inserir um elemento no final de uma lista (cont.)

- Caso o elemento a ser adicionado na lista seja uma lista, o `append` cria uma sublista
- Exemplos:

```
lista_orig = [45, 32, 88]
```

```
lista_orig.append("gato")
```

```
lista_orig.append(["cachorro"])
```

```
print(lista_orig) # saída: [45, 32, 88, 'gato', ['cachorro']]
```

```
lista_orig = lista_orig + ["sapo"]
```

```
print(lista_orig) # saída: [45, 32, 88, 'gato', ['cachorro'], 'sapo']
```

Inserir um elemento no final de uma lista (cont.)

- É possível acrescentar um item no final da lista usando o operador **+** de concatenação
 - É necessário ter cuidado!!! Com a concatenação uma nova lista é criada e o id da nova lista é diferente do id da lista original
- Exemplos:

```
>>> lista_orig = ['gato', 'pato', 'rato']
```

```
>>> print(id(lista_orig))
```

47528176

```
>>> lista_orig = lista_orig + ['sapo']
```

```
>>> print(id(lista_orig))
```

47403496

Inserir um elemento em determinada posição de uma lista

- A função **insert()** adiciona um novo elemento em uma posição qualquer de uma lista.
 - Sintaxe: `lista.insert(parâmetro1, parâmetro2)`, onde `parâmetro1` é a posição (índice) de inserção na lista e `parâmetro2` é o elemento a ser inserido naquela posição.
- Exemplo: inserindo a string 'morte' na posição de índice **1** da lista a seguir:

```
cavaleiros = ['guerra', 'fome', 'peste']  
cavaleiros.insert(1, 'morte')  
print(cavaleiros)  
['guerra', 'morte', 'fome', 'peste']
```

Removendo um elemento de uma lista

- O comando **del** remove um elemento de uma lista usando a sua posição

- Exemplo:

```
cavaleiros = ['guerra', 'morte', 'fome', 'peste']
```

```
del cavaleiros[2]
```

```
print(cavaleiros)
```

```
['guerra', 'morte', 'peste']
```

- O comando **del** também manipula índices negativos e produz um erro de execução se o índice estiver fora do intervalo da lista

Replicando elementos de uma lista (operador repetição)

- O operador `*` repete os itens em uma lista um dado número de vezes
- Exemplos:

```
>>>print([0] * 4)  
[0, 0, 0, 0]
```

```
>>>print([1, 2, "Olá, tudo bem? " * 2]  
[1, 2, 'Olá, tudo bem? Olá, tudo bem? ']
```

Observações Importantes

- Os operadores de concatenação e repetição criam **novas listas** (diferentes *ids*) a partir dos elementos das listas dadas
- Por exemplo, se você concatena uma lista de 2 itens com uma lista de 4 itens, você obterá uma nova lista com 6 itens e não uma lista com duas sublistas
- Similarmente, repetição de uma lista de 2 itens 4 vezes resultará em uma lista com 8 itens

Observações Importantes (cont.)

- Exemplo:

```
lista_orig=[0]
print(lista_orig)# imprime [0]

print(id(lista_orig)) # imprime 47492352

lista_orig= lista_orig * 4
print(lista_orig) )# imprime [0, 0, 0, 0]

print(id(lista_orig)) # imprime 47665560
```

.

Exercício de fixação

1. Faça um programa que crie uma lista com 10 números inteiros fornecidos pelo usuário. Após criar a lista, o programa deverá imprimir:
 - O maior elemento da lista
 - O menor elemento da lista
 - A soma de todos os elementos da lista
 - Os elementos ímpares
 - Os elementos maiores do que 18

Obs: não use funções prontas.

Fatias de Listas

- A operação de fatiar (*slice*) pode ser aplicada sobre listas
- O primeiro índice indica o ponto do início da fatia e o segundo índice é um a mais do final da fatia (o elemento com esse índice não faz parte da fatia)

- Exemplos:

```
lista = ['a', 'b', 'c', 'd', 'e', 'f']  
print(lista[1:3]) # imprime ['b', 'c']  
print(lista[:4]) #imprime ['a', 'b', 'c', 'd']  
print(lista[3:]) #imprime ['d', 'e', 'f']  
print(lista[:]) #imprime ['a', 'b', 'c', 'd', 'e', 'f']
```

Inserindo elementos com o operador slice

- Podemos inserir elementos em uma lista espremendo-os em uma fatia vazia na posição desejada

- Exemplo:

```
lista = ['a', 'd', 'f']  
print(lista) # imprime ['a', 'd', 'f']
```

```
lista[1:1] = ['b', 'c']  
print(lista) # imprime ['a', 'b', 'c', 'd', 'f']
```

```
lista[4:4] = ['e']  
print(lista) # imprime ['a','b','c','d','e','f']
```

Removendo elementos com o operador slice (cont.)

- Exemplos:

```
lista = ['a', 'b', 'c', 'd', 'e', 'f']  
del lista[1:5]  
print(lista)  
['a', 'f']
```

- Pode-se remover elementos de uma lista atribuindo a lista vazia a eles:

- Exemplo:

```
uma_lista = ['a', 'b', 'c', 'd', 'e', 'f']  
uma_lista[1:3] = ['x', 'y']  
print(uma_lista) #Saída: ['a', 'x', 'y', 'b', 'c', 'd', 'e', 'f']  
uma_lista[1:3] = [ ]  
print(uma_lista) #Saída: ['a', 'd', 'e', 'f']
```

Encontrando a posição de um elemento qualquer de uma lista

- A função `index()` é usada para encontrar a posição de um elemento qualquer em uma lista.
- Por exemplo, para encontrar a posição do elemento 'peste' na lista a seguir, faça:

```
>>>cavaleiros = ['guerra', 'morte', 'fome', 'peste']  
>>>cavaleiros.index('peste')
```

2

Criando listas de inteiros – operador range

- Listas que contém números inteiros consecutivos são bastante comuns.
- O operador **range** fornece uma maneira simples de criar uma lista de inteiros.
- Exemplos:

```
>>> L = list(range(5,13))
```

```
>>> print(L)
```

```
[5, 6, 7, 8, 9, 10, 11, 12]
```

- O operador **range** pega dois argumentos (dois inteiros) e devolve uma lista que contém todos os inteiros do primeiro até o segundo, incluindo o primeiro mas não incluindo o segundo!

Criando listas de inteiros – operador range (cont.)

- Com um argumento apenas, **range** cria uma lista que inicia em 0.

- Exemplo:

```
>>> L = list(range(10))
```

```
>>> print(L)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

- Se houver um terceiro argumento, ele especifica o “espaço” entre os elementos sucessivos. Exemplo:

```
>>> L = list(range(2, 23, 3))
```

```
>>> print(L)
```

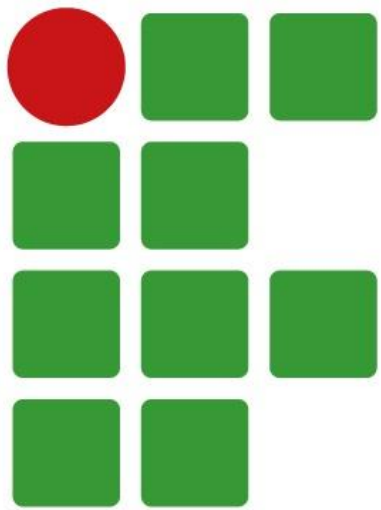
```
[2, 5, 8, 11, 14, 17, 20]
```

Exercícios

1. Gere uma lista contendo os múltiplos de 3 entre 1 e 150.
2. Crie um programa que leia inicialmente uma sequência de N notas de alunos fornecidas pelo usuário e ao final mostre a sequência e sua média aritmética. Defina um critério de parada para a entrada de notas pelo usuário.
3. Crie um programa que leia inicialmente uma sequência de N números inteiros e ao seu final mostre a sequência original, a soma de seus elementos que forem pares e a multiplicação dos elementos que forem ímpares.
4. Crie um programa que leia inicialmente uma sequência de N números inteiros. Depois, o programa deve gerar e mostrar 2 novas listas a partir da primeira: uma sem repetição de elementos e outra com os elementos que se repetem na lista original.

Exercícios

5. Crie um programa que leia inicialmente duas sequências de N elementos cada uma e ao final mostre as duas sequências originais e a sequência resultante da soma de seus elementos. Exemplo:
 $a=[5, 9, 0]$ $b=[12, 34, 101]$ $soma=[17, 43, 101]$
6. Crie um programa que dada uma sequência de N elementos fornecidos pelo usuário mostre a sequência original e a sequência elevada ao cubo.
7. Crie um programa que leia inicialmente uma sequência de N números inteiros fornecidos pelo usuário e mostre ao final da leitura a sequência original e a sequência invertida.
8. Crie um programa que leia inicialmente uma sequência de N elementos inteiros positivos fornecidos pelo usuário e substitua seus elementos de valor ímpar por -1 e os pares por +1. Ao final imprima a sequência original e a sequência alterada.



INSTITUTO FEDERAL

São Paulo

Câmpus São Carlos

Listas como elementos de outra Lista

- Uma lista pode conter outras listas como elementos
- Uma lista de listas é dita aninhada (*nested*) e a lista mais interna é chamada frequentemente de sublista (*sublist*)
- Exemplos:

```
>>>lista_mista = ["ola", 2.0, 5*2, [10, 20]]  
>>>lista = ['O carro','peixe',123,111]  
>>>nova_lista = ['pedra',lista]
```

```
>>>print(lista_mista)  
['ola', 2.0, 10, [10, 20]]
```

```
>>> print(lista)  
['O carro', 'peixe', 123, 111]
```

```
>>> print(nova_lista)  
['pedra', ['O carro', 'peixe', 123, 111]]
```